

**LAPORAN TUGAS 6**

**SISTEM OPERASI**



**Dosen Pengampu:**

**Triawan Adi Cahyanto, M.Kom**

**Disusun Oleh:**

**Yoanre Louis A.H.P (2410651039)**

**PRODI TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER**

**2025**

## DAFTAR ISI

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>BAB 1 .....</b>                                        | <b>3</b>  |
| <b>PENDAHULUAN .....</b>                                  | <b>3</b>  |
| <b>1.1 LATAR BELAKANG .....</b>                           | <b>3</b>  |
| <b>1.2 RUMUSAN MASALAH.....</b>                           | <b>3</b>  |
| <b>1.3 TUJUAN .....</b>                                   | <b>4</b>  |
| <b>BAB II .....</b>                                       | <b>5</b>  |
| <b>SOAL TUGAS .....</b>                                   | <b>5</b>  |
| <b>2.1 SOAL.....</b>                                      | <b>5</b>  |
| <b>BAB III.....</b>                                       | <b>6</b>  |
| <b>PEMBAHASAN .....</b>                                   | <b>6</b>  |
| <b>3.1 ANALISIS DEADLOCK.....</b>                         | <b>6</b>  |
| <b>3.2 IMPLEMENTASI PENCEGAHAN.....</b>                   | <b>7</b>  |
| <b>3.2.1 MEMODIFIKASI PROGRAM DEADLOCK_SIMPLE.C .....</b> | <b>7</b>  |
| <b>3.3 TESTING &amp; OUTPUT .....</b>                     | <b>13</b> |
| <b>3.4 REALCASE.....</b>                                  | <b>19</b> |
| <b>BAB IV .....</b>                                       | <b>23</b> |
| <b>KESIMPULAN .....</b>                                   | <b>23</b> |
| <b>4.1 KESIMPULAN .....</b>                               | <b>23</b> |
| <b>DAFTAR PUSTAKA .....</b>                               | <b>24</b> |

# **BAB 1**

## **PENDAHULUAN**

### **1.1 LATAR BELAKANG**

Dalam pengembangan sistem komputasi modern yang bersifat konkuren, manajemen resource yang efisien menjadi tantangan kritis. Sistem operasi harus dapat mengelola alokasi resource ke multiple thread atau proses tanpa menimbulkan kondisi deadlock. Deadlock merupakan situasi dimana dua atau lebih proses saling menunggu resource yang dipegang oleh proses lain, mengakibatkan kebuntuan eksekusi yang mengganggu stabilitas sistem. Fenomena ini memenuhi empat kondisi necessary condition: mutual exclusion, hold and wait, no preemption, dan circular wait. Dalam praktik pemrograman konkuren, kasus sederhana seperti akses bersamaan ke dua mutex dapat dengan mudah memicu deadlock, sebagaimana ditunjukkan dalam program `deadlock_simple.c`. Oleh karena itu, pemahaman mendalam mengenai mekanisme pencegahan deadlock seperti lock ordering, try-lock, dan algoritma penjadwalan resource yang aman seperti Banker's Algorithm menjadi kompetensi fundamental. Laporan tugas ini disusun untuk menganalisis penyebab, mengimplementasikan solusi pencegahan, dan mensimulasikan deadlock dalam konteks nyata, guna membangun sistem yang lebih robust dan terhindar dari kondisi kebuntuan yang merugikan.

### **1.2 RUMUSAN MASALAH**

1. Mengapa program `deadlock_simple.c` mengalami deadlock dan kondisi apa saja yang terpenuhi?
2. Bagaimana cara memodifikasi program untuk mencegah deadlock menggunakan teknik Lock Ordering dan Try-Lock, serta apa perbandingan kelebihan dan kekurangan kedua teknik tersebut?
3. Bagaimana mengimplementasikan Banker's Algorithm untuk mengelola alokasi resource yang aman dan mensimulasikan pencegahan deadlock pada kasus nyata transfer dana antar rekening?

### **1.3 TUJUAN**

1. Menganalisis penyebab terjadinya deadlock pada program `deadlock_simple.c` serta mengidentifikasi keempat kondisi deadlock yang terpenuhi.
2. Mengimplementasikan dan membandingkan dua teknik pencegahan deadlock, yaitu Lock Ordering dan Try-Lock dengan Retry, beserta analisis performanya.
3. Membuat program simulasi yang mengimplementasikan Banker's Algorithm untuk manajemen resource dan kasus nyata transfer dana dengan pencegahan deadlock.

## **BAB II**

### **SOAL TUGAS**

#### **2.1 SOAL**

##### **Tugas 1: Analisis Deadlock**

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

##### **Tugas 2: Implementasi Pencegahan**

Modifikasi program `deadlock_simple.c` menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

##### **Tugas 3: Banker's Algorithm**

Buat program baru dengan data: 4 proses

- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

##### **Tugas 4: Kasus Nyata (Real Case)**

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

## BAB III

### PEMBAHASAN

#### 3.1 ANALISIS DEADLOCK

```
dany@LAPTOP-GRJL2V1D:~/praktikum6$ nano deadlock_simple.c
dany@LAPTOP-GRJL2V1D:~/praktikum6$ gcc deadlock_simple.c -o deadlock_simple -lpthread
dany@LAPTOP-GRJL2V1D:~/praktikum6$ ./deadlock_simple
=== SIMULASI DEADLOCK ===

Thread 1: Mencoba mengambil Lock 1...
Thread 1: Berhasil dapat Lock 1!
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Berhasil dapat Lock 2!
Thread 2: Mencoba mengambil Lock 1...
Thread 1: Mencoba mengambil Lock 2...
|
```

#### Mengapa program mengalami deadlock?

Program mengalami deadlock karena kedua thread saling menunggu resource (mutex) yang sedang dipegang oleh thread lainnya, dan tidak ada satupun yang akan melepaskannya. Urutannya sebagai berikut:

1. Thread 1 berhasil mengambil Lock 1
2. Thread 2 berhasil mengambil Lock 2
3. Setelah sleep (1)
  - Thread 1 mencoba mengambil Lock 2, tetapi Lock 2 sedang dipegang Thread 2 → Thread 1 menunggu
  - Thread 2 mencoba mengambil Lock 1, tetapi Lock 1 sedang dipegang Thread 1 → Thread 2 menunggu

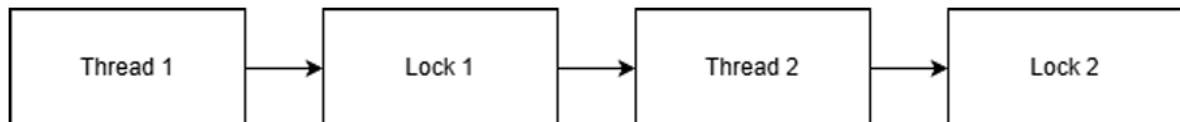
Masalah:

- Thread 1 menunggu Thread 2
- Thread 2 menunggu Thread 1
- Tidak ada yang bisa melanjutkan eksekusi → kondisi deadlock permanen.
- Masing-masing thread mengambil lock dalam urutan yang berbeda, sehingga terjadi circular wait.

## Kondisi deadlock apa saja yang terpenuhi?

1. MUTEX (Mutual Execution): Deadlock terjadi karena mutex hanya dapat digunakan satu thread pada satu waktu, sehingga kedua thread tidak bisa mengakses resource secara bersamaan.
2. Hold and Wait: Setiap thread sudah memegang satu mutex tetapi tetap menunggu mutex lain, menyebabkan mereka terjebak sambil mempertahankan resource.
3. No Preemption: Mutex tidak dapat direbut secara paksa oleh thread lain, sehingga resource yang sudah dipegang tidak bisa dilepas sebelum thread selesai.
4. Circular Wait: Terjadi siklus tunggu: Thread 1 menunggu lock2 dan Thread 2 menunggu lock1, membentuk lingkaran yang tidak bisa diselesaikan.

## Gambar diagram resource allocation graph-nya



## 3.2 IMPLEMENTASI PENCEGAHAN

### 3.2.1 MEMODIFIKASI PROGRAM DEADLOCK\_SIMPLE.C

#### LOCK ORDERING:

```
dany@LAPTOP-GRJL2V1D:~/praktikum6$ nano deadlock_simple.c
dany@LAPTOP-GRJL2V1D:~/praktikum6$ gcc deadlock_simple.c -o deadlock_simple -lpthread
dany@LAPTOP-GRJL2V1D:~/praktikum6$ ./deadlock_simple
=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===
Thread 1: Mengambil lock secara terurut...
Thread 1: Menggunakan resource...
Thread 2: Mengambil lock secara terurut...
Thread 1: Selesai.
Thread 2: Menggunakan resource...
Thread 2: Selesai.
Program selesai tanpa deadlock.
dany@LAPTOP-GRJL2V1D:~/praktikum6$ |
```

#### KODE C:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void acquire_locks_ordered() {
    pthread_mutex_lock(&lock1);
    pthread_mutex_lock(&lock2);
}

void release_locks_ordered() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}

void* thread1_func(void* arg) {
    printf("Thread 1: Mengambil lock secara teratur...\n");
    acquire_locks_ordered();

    printf("Thread 1: Menggunakan resource...\n");
    sleep(1);

    release_locks_ordered();
    printf("Thread 1: Selesai.\n");
    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: Mengambil lock secara teratur...\n");
    acquire_locks_ordered();

    printf("Thread 2: Menggunakan resource...\n");
    sleep(1);

    release_locks_ordered();
    printf("Thread 2: Selesai.\n");
    return NULL;
}
```



```

}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: LOCK ORDERING ===\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Program selesai tanpa deadlock.\n");
    return 0;
}

```

### TRY – LOCK DENGAN ENTRY:

```

dany@LAPTOP-GRJL2V1D:~/praktikum6$ nano deadlock_simple.c
dany@LAPTOP-GRJL2V1D:~/praktikum6$ gcc deadlock_simple.c -o deadlock_simple -lpthread
dany@LAPTOP-GRJL2V1D:~/praktikum6$ ./deadlock_simple
=== PENCEGAHAN DEADLOCK: TRY-LOCK ===
Thread 1: Dapat Lock1, mencoba Lock2...
Thread 1: Gagal dapat Lock2, melepas Lock1...
Thread 2: Dapat Lock2, mencoba Lock1...
Thread 2: Berhasil dapat kedua lock!
Thread 2 selesai (retry=0)
Thread 1: Dapat Lock1, mencoba Lock2...
Thread 1: Berhasil dapat kedua lock!
Thread 1 selesai (retry=1)
Program selesai tanpa deadlock.
dany@LAPTOP-GRJL2V1D:~/praktikum6$ |

```

### KODE C:

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

```

```

void* thread1_func(void* arg) {
    int retry = 0;

    while (1) {
        pthread_mutex_lock(&lock1);
        printf("Thread 1: Dapat Lock1, mencoba Lock2...\n");

        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1: Berhasil dapat kedua lock!\n");
            sleep(1);

            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        }

        printf("Thread 1: Gagal dapat Lock2, melepas Lock1...\n");
        pthread_mutex_unlock(&lock1);
        retry++;
        usleep(100000);
    }

    printf("Thread 1 selesai (retry=%d)\n", retry);
    return NULL;
}

void* thread2_func(void* arg) {
    int retry = 0;

    while (1) {
        pthread_mutex_lock(&lock2);
        printf("Thread 2: Dapat Lock2, mencoba Lock1...\n");
    }
}

```

```

        if(pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2: Berhasil dapat kedua lock!\n");
            sleep(1);

            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        }

        printf("Thread 2: Gagal dapat Lock1, melepas Lock2...\n");
        pthread_mutex_unlock(&lock2);
        retry++;
        usleep(120000);
    }

    printf("Thread 2 selesai (retry=%d)\n", retry);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== PENCEGAHAN DEADLOCK: TRY-LOCK ===\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Program selesai tanpa deadlock.\n");
    return 0;
}

```

```
}
```

### PERBANDINGAN KEDUA METODE:

| Aspek                 | Lock Ordering                                   | Try-Lock + Retry                         |
|-----------------------|-------------------------------------------------|------------------------------------------|
| Cara kerja            | Semua thread ambil lock dalam urutan yang sama  | Thread mencoba lock; jika gagal, ulangi  |
| Kemungkinan deadlock  | 0% (sangat aman)                                | 0% (resource dilepas saat gagal)         |
| Performa              | Cepat dan stabil                                | Bisa lambat jika retry banyak            |
| Kompleksitas kode     | Sederhana                                       | Lebih rumit (loop, retry, sleep)         |
| Risiko starvation     | Hampir tidak ada                                | Ada kemungkinan satu thread kalah terus  |
| Kapan cocok digunakan | Jika urutan lock bisa ditentukan dengan mudah   | Jika urutan lock tidak bisa distandarkan |
| Kelebihan utama       | Simple, deterministik                           | Fleksibel, tidak butuh urutan lock       |
| Kekurangan            | Tidak cocok jika resource tidak dapat diurutkan | CPU usage meningkat jika retry sering    |

### 3.3 BANKER'S ALGORITHM

```
dany@LAPTOP-GRJL2V1D:~/praktikum6$ nano bankers_algorithm.c
dany@LAPTOP-GRJL2V1D:~/praktikum6$ gcc bankers_algorithm.c -o bankers_algorithm
dany@LAPTOP-GRJL2V1D:~/praktikum6$ ./bankers_algorithm
=== BANKER'S ALGORITHM (4 PROSES) ===

=== STATUS SISTEM ===
Available: 5 4 3

Allocation:
P0: 1 0 0
P1: 1 1 0
P2: 1 2 1
P3: 0 0 1

Need:
P0: 3 3 2
P1: 1 1 2
P2: 2 1 2
P3: 4 2 0

=== CEK KEAMANAN SISTEM ===
P0 dapat dijalankan.
P1 dapat dijalankan.
P2 dapat dijalankan.
P3 dapat dijalankan.
[SAFE] Sistem dalam keadaan aman.
Safe sequence: P0 P1 P2 P3

=== REQUEST DARI P1 ===
Meminta: 1 0 1

=== CEK KEAMANAN SISTEM ===
P0 dapat dijalankan.
P1 dapat dijalankan.
P2 dapat dijalankan.
P3 dapat dijalankan.
[SAFE] Sistem dalam keadaan aman.
Safe sequence: P0 P1 P2 P3
[DITERIMA] Request aman dan dialokasikan.
```

```
=== STATUS SISTEM ===
Available: 4 4 2

Allocation:
P0: 1 0 0
P1: 2 1 1
P2: 1 2 1
P3: 0 0 1

Need:
P0: 3 3 2
P1: 0 1 1
P2: 2 1 2
P3: 4 2 0

=== REQUEST DARI P2 ===
Meminta: 3 3 0
[DITOLAK] Request melebihi need!

=== PROGRAM SELESAI ===
dany@LAPTOP-GRJL2V1D:~/praktikum6$ |
```

KODE C:

```
#include <stdio.h>
#include <stdbool.h>

#define P 4 // jumlah proses
#define R 3 // jumlah jenis resource

// Data sistem
```

```

int available[R] = {5, 4, 3};

int maximum[P][R] = {
    {4, 3, 2},
    {2, 2, 2},
    {3, 3, 3},
    {4, 2, 1}
};

int allocation[P][R] = {
    {1, 0, 0},
    {1, 1, 0},
    {1, 2, 1},
    {0, 0, 1}
};

int need[P][R];

// Hitung need = max - allocation
void calculate_need() {
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = maximum[i][j] - allocation[i][j];
}

void print_status() {
    printf("\n=== STATUS SISTEM ===\n");

    printf("Available: ");
    for (int i = 0; i < R; i++) printf("%d ", available[i]);
    printf("\n\nAllocation:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
    }
}

```

```

        for (int j = 0; j < R; j++) printf("%d ", allocation[i][j]);
        printf("\n");
    }

    printf("\nNeed:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
}

// Cek apakah state aman
bool is_safe() {
    int work[R];
    bool finish[P] = {false};
    int safe_seq[P];
    int count = 0;

    for (int i = 0; i < R; i++)
        work[i] = available[i];

    printf("\n=== CEK KEAMANAN SISTEM ===\n");

    while (count < P) {
        bool found = false;

        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool can_run = true;

                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {

```

```

        can_run = false;
        break;
    }
}

if (can_run) {
    printf("P%d dapat dijalankan.\n", i);

    for (int j = 0; j < R; j++)
        work[j] += allocation[i][j];

    safe_seq[count++] = i;
    finish[i] = true;
    found = true;
}
}
}

if (!found) {
    printf("[UNSAFE] Tidak ditemukan safe sequence!\n");
    return false;
}
}

printf("[SAFE] Sistem dalam keadaan aman.\n");
printf("Safe sequence: ");
for (int i = 0; i < P; i++)
    printf("P%d ", safe_seq[i]);
printf("\n");

return true;
}

```



```

// Fungsi request resource
bool request_resources(int process, int request[]) {
    printf("\n=== REQUEST DARI P%d ===\n", process);
    printf("Meminta: ");
    for (int i = 0; i < R; i++) printf("%d ", request[i]);
    printf("\n");

    // 1. Cek request <= need
    for (int i = 0; i < R; i++) {
        if (request[i] > need[process][i]) {
            printf("[DITOLAK] Request melebihi need!\n");
            return false;
        }
    }

    // 2. Cek request <= available
    for (int i = 0; i < R; i++) {
        if (request[i] > available[i]) {
            printf("[DITOLAK] Resource tidak tersedia!\n");
            return false;
        }
    }

    // 3. Coba alokasi sementara
    for (int i = 0; i < R; i++) {
        available[i] -= request[i];
        allocation[process][i] += request[i];
        need[process][i] -= request[i];
    }

    // 4. Cek safe state
    if (is_safe()) {
        printf("[DITERIMA] Request aman dan dialokasikan.\n");
    }
}

```

```

    return true;
}

// 5. Jika unsafe → rollback
printf("[DITOLAK] Request membuat sistem menjadi unsafe! Rollback...\n");
for (int i = 0; i < R; i++) {
    available[i] += request[i];
    allocation[process][i] -= request[i];
    need[process][i] += request[i];
}

return false;
}

int main() {
    printf("=== BANKER'S ALGORITHM (4 PROSES) ===\n");

    calculate_need();
    print_status();

    is_safe();

    // Contoh request aman
    int req1[R] = {1, 0, 1};
    request_resources(1, req1);
    print_status();

    // Contoh request tidak aman
    int req2[R] = {3, 3, 0};
    request_resources(2, req2);

    printf("\n=== PROGRAM SELESAI ===\n");
    return 0;
}

```

```
}
```

Program Banker's Algorithm ini dibuat untuk mensimulasikan manajemen resource pada empat proses dan tiga jenis resource dengan nilai available awal [5, 4, 3]. Sistem menghitung matriks need berdasarkan selisih antara maximum dan allocation, lalu mengevaluasi apakah suatu request resource aman dengan mengecek kondisi safe state menggunakan simulasi urutan eksekusi proses. Jika request menghasilkan status aman, alokasi diterima; namun jika menyebabkan unsafe state, sistem melakukan rollback agar tetap terhindar dari potensi deadlock.

### 3.4 REALCASE

```
PROGRAM SELESAI
dany@LAPTOP-GRJL2V1D:~/praktikum6$ nano deadlock_prevention.c
dany@LAPTOP-GRJL2V1D:~/praktikum6$ gcc deadlock_prevention.c -o deadlock_prevention
dany@LAPTOP-GRJL2V1D:~/praktikum6$ ./deadlock_prevention
SIMULASI TRANSFER TANPA DEADLOCK

Saldo awal:
Rek1 = 1000
Rek2 = 2000

A mulai transfer 300 dari Rek1 ke Rek2...
B mulai transfer 500 dari Rek2 ke Rek1...
A selesai transfer.
B selesai transfer.

Saldo akhir:
Rek1 = 1200
Rek2 = 1800

PROGRAM SELESAI TANPA DEADLOCK
dany@LAPTOP-GRJL2V1D:~/praktikum6$ |
```

KODE C:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int saldo1 = 1000;
int saldo2 = 2000;

pthread_mutex_t lock_rek1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock_rek2 = PTHREAD_MUTEX_INITIALIZER;

void lock_in_order(pthread_mutex_t *first, pthread_mutex_t *second) {
    pthread_mutex_lock(first);
```

```
pthread_mutex_lock(second);
}

void unlock_in_order(pthread_mutex_t *first, pthread_mutex_t *second) {
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}

void* transfer_A(void* arg) {
    printf("A mulai transfer 300 dari Rek1 ke Rek2...\n");

    lock_in_order(&lock_rek1, &lock_rek2);

    saldo1 -= 300;
    saldo2 += 300;
    sleep(1);

    printf("A selesai transfer.\n");

    unlock_in_order(&lock_rek1, &lock_rek2);

    return NULL;
}

void* transfer_B(void* arg) {
    printf("B mulai transfer 500 dari Rek2 ke Rek1...\n");

    lock_in_order(&lock_rek1, &lock_rek2);

    saldo2 -= 500;
    saldo1 += 500;
    sleep(1);
```

```

printf("B selesai transfer.\n");

unlock_in_order(&lock_rek1, &lock_rek2);

return NULL;
}

int main() {
    pthread_t A, B;

    printf("SIMULASI TRANSFER TANPA DEADLOCK\n\n");
    printf("Saldo awal:\nRek1 = %d\nRek2 = %d\n\n", saldo1, saldo2);

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    printf("\nSaldo akhir:\nRek1 = %d\nRek2 = %d\n", saldo1, saldo2);
    printf("\nPROGRAM SELESAI TANPA DEADLOCK\n");

    return 0;
}

```

Program ini mensimulasikan kasus nyata dua orang yang melakukan transfer uang secara bersamaan antar dua rekening, yang berpotensi menyebabkan deadlock jika kedua thread mengunci rekening dalam urutan berbeda. Untuk mencegah deadlock, program menggunakan teknik lock ordering, yaitu memastikan semua thread selalu mengunci resource dalam urutan yang sama, yakni Rekening 1 terlebih dahulu kemudian Rekening 2. Dengan cara ini, kondisi circular wait tidak mungkin terjadi karena tidak ada thread yang mengambil lock secara terbalik. Setiap transaksi berlangsung dalam critical section yang terlindungi sehingga

perubahan saldo aman dan konsisten. Setelah kedua thread selesai, program menampilkan saldo akhir dan memastikan proses berlangsung tanpa deadlock.

## **BAB IV**

### **KESIMPULAN**

#### **4.1 KESIMPULAN**

Laporan ini berhasil menganalisis, mengimplementasikan, dan mensimulasikan berbagai aspek deadlock dalam sistem operasi. Dari analisis awal, terkonfirmasi bahwa program `deadlock_simple.c` mengalami deadlock karena memenuhi keempat kondisi necessary condition, yaitu mutual exclusion, hold and wait, no preemption, dan circular wait. Implementasi teknik pencegahan dengan Lock Ordering terbukti efektif mencegah deadlock dengan cara yang deterministik dan sederhana, sementara Try-Lock menawarkan fleksibilitas dengan mekanisme coba-ulang. Banker's Algorithm berhasil diimplementasikan untuk memastikan sistem berada dalam state yang aman sebelum mengalokasikan resource. Simulasi kasus nyata transfer dana juga membuktikan bahwa deadlock dapat dicegah dengan penerapan Lock Ordering yang konsisten. Secara keseluruhan, tugas ini menegaskan bahwa pemahaman dan penerapan teknik manajemen resource yang tepat adalah kunci untuk membangun sistem konkuren yang bebas dari deadlock.

## **DAFTAR PUSTAKA**

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.